Centro Universitário de Excelência
Sistemas de Informação

Dendê Eventos Api-Madri

Autores:
Andrew Oliveira Neves
Filipe Oliveira Santos
Guilherme Rodrigues França da Rocha
Gustavo Santos França
José Raimundo de Melo Filho

Vitória da Conquista, 2026

Agenda

OBJETIVO DO PROJETO

Desenvolver uma API REST para o aplicativo Dendê Eventos, permitindo o gerenciamento completo de:

✓ Usuários comuns
✓ Organizadores de eventos
✓ Eventos
✓ Ingressos

Baseado nas 15 User Stories fornecidas

1. Introdução/Visão Geral

2. Arquitetura do Sistema

3. Modelagem de Dados (Diagrama de Classes)

4. Endpoints Implementados

5. Desafios e Soluções

6. Considerações Finais

Introdução

- **CONTEXTO**

  O Dendê Eventos é um aplicativo para divulgação, organização e gestão de eventos culturais, acadêmicos e sociais, conectando organizadores e público de forma simples e eficiente.

  Neste projeto, desenvolvemos uma API REST que dá vida às funcionalidades do sistema, permitindo que usuários comuns possam comprar ingressos e organizadores possam criar e gerenciar seus eventos, seguindo as regras de negócio estabelecidas nas 15 User Stories fornecidas.

Visão Geral do Sistema

- API MADRI é uma API REST desenvolvida para gerenciar eventos de uma forma estruturada e organizada

O Sistema permite:

- Cadastro e gerenciamento de usuários e organizadores;
- Criação, ativação e controle de eventos;
- Compra e cancelamento de Ingressos;
- Controle de disponibilidade e capacidade.

Arquitetura Sistema

A arquitetura do sistema foi planejada com o objetivo de garantir organização, escalabilidade e separação de responsabilidades. O projeto foi estruturado seguindo o modelo em camadas, permitindo que cada parte do sistema desempenhe uma função específica dentro da aplicação.

- **Camada de Controller**

Responsável por receber as requisições do cliente, interpretar os dados enviados e retornar as respostas apropriadas. Essa camada define os endpoints da aplicação e faz a ponte entre o usuário e a lógica de negócio.

- **Camada de Service**

Contém as regras de negócio do sistema. Nessa camada são implementadas as validações, cálculos e verificações necessárias para que o sistema funcione corretamente.

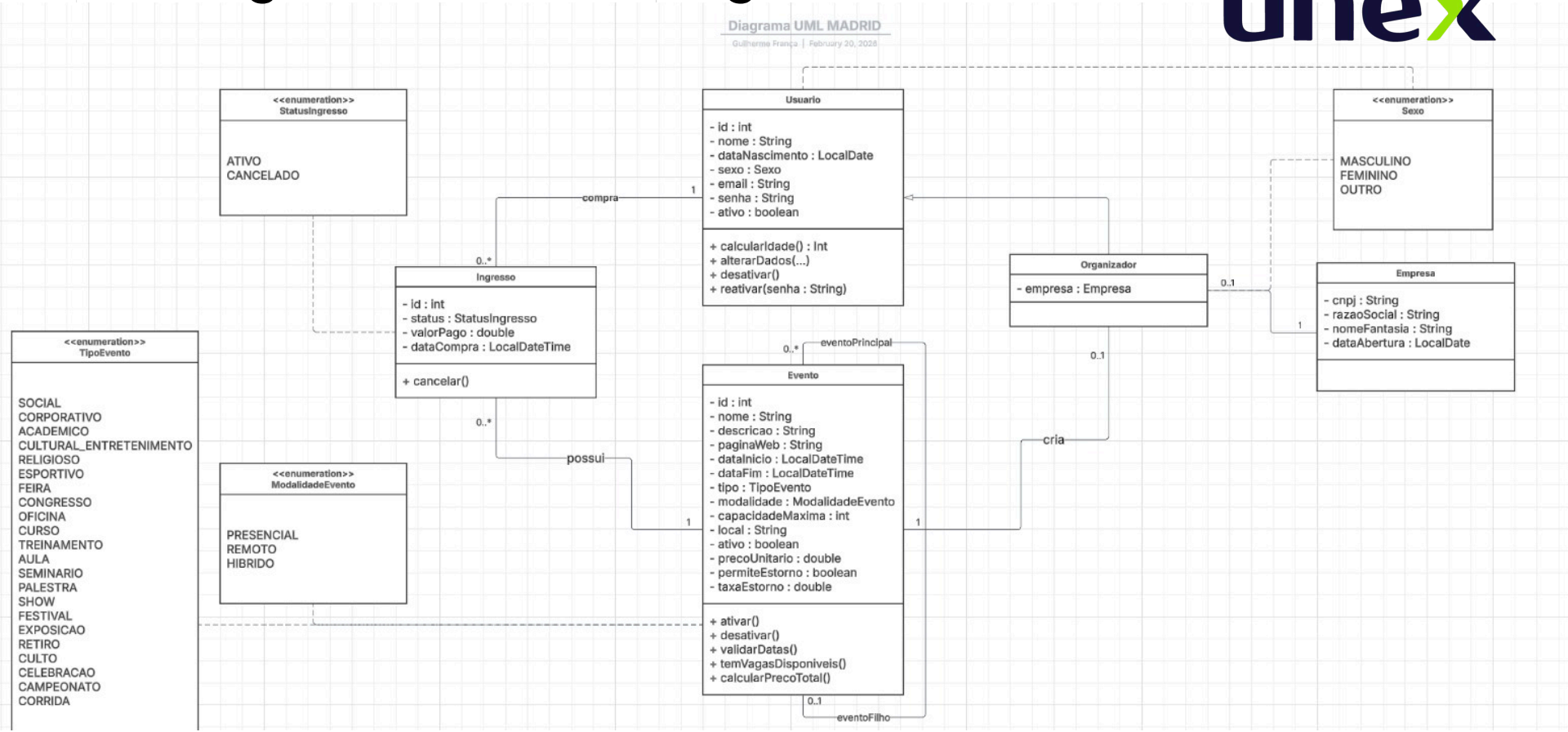
- **Camada de Repository**

Responsável pela comunicação com o banco de dados. Essa camada realiza operações de inserção, consulta, atualização e remoção de dados.

- **Camada de Model (Entidades)**

Representa as classes do sistema, conforme definido no diagrama de classes. Essas classes correspondem às tabelas do banco de dados e armazenam as informações manipuladas pela aplicação.

Modelagem de Dados: Diagrama de Classe



Endpoints

US 1 **Cadastro de Usuário Comum** POST /Usuários	**US 6** **Reativar** Patch /organizadores/ {id}/{status}	**US 11** **Listar eventos do organizador** GET/organizadores/{id}/eventos
US 2 **Cadastrar Organizador** POST /organizadores	**US 7** **Cadastrar Evento** POST /organizadores/{id}/evento	**US 12** **Feed de Eventos** GET /eventos
US 3 **Alterar Perfil** PUT /usuários/{id}	**US 8** **Alterar Evento** PUT /organizadores/{orgid}/eventos/{eventoId}	**US 13** **Comprar Ingresso** POST /organizadores/ {orgid}/ eventos/{evid}/ingresos
US 4 **Visualizar Perfil** GET/usuarios/{id}	**US 9** **Ativar Evento** PATCH /organizadores/{orgid}/ eventos/{eventoID}/{status}	**US 14** **Cancelar Ingresso** POST /usuáros/ {id}/ ingresos/{ingressoId}
US 5 **Desativar** PATCH /organizadores/ {id}/{status}	**US 10** **Desativar Evento** PATCH/organizadores/{orgid}/eventos/{ eventoID}/{status}	**US 15** **Listar Ingressos do Usuário** GET /usuários/{id}ingressos

Desafios e Soluções:

DESAFIO : Não expor senha nas respostas

- Problema: GET retornava senha no toString()
- Solução: DTOs específicos para resposta
- Código: UsuarioResponseDTO não contém campo senha

DESAFIO: Testar o sistema para validar endpoints no postman

- Problema: Para muitos requisitos do sistema, era necessário o uso de recursos que não dominávamos
- Solução: Pesquisas visando aprender e aplicar as funcionalidades do Java

Projetar rotas RESTfull intuitivas e alinhadas às 15 User Stories

- Problema: Modelar as rotas de maneira eficiente para atender a todos os requisitos do sistema
- Solução: Aprender os conceitos da camada Controllers e analisar a Collection: dende_softhouse.postman_collection.json

Considerações finais

CONCLUSÃO

Para a finalização deste projeto, foi imprescindível o estudo de técnicas primordiais de Orientação a Objetos e Arquitetura de Software que certamente fomentaram nossa base teórica (e prática, por que não dizer) para projetos futuros, nos conscientizando sobre como deve ser o planejamento, estruturação e desenvolvimento de uma aplicação.

Link Do Repositorio:

https://github.com/Gussffy/dende-eventos-api-madri

Referências

- SIERRA, Kathy; BATES, Bert. Use a cabeça! Java. 2. ed. Rio de Janeiro: Alta Books, 2010.